Artha: A Domain Ontology-Driven Agentic Framework for LLM-Based Personal Finance Reasoning

Tejas Katika
Department of Computer Science and Engineering
University of North Texas, Denton, TX, USA
tejas.katika@my.unt.edu

***Abstract*—Personal financial data is transactional, not linguistic. Merchant strings such as *SQ *WHOLE FOODS 982* carry no semantic structure; dollar amounts lack categorical context; timestamps bear no relationship to budgets or goals. We present Artha (Sanskrit: *artha*—wealth, meaning, purpose), a framework that pre-computes a typed domain ontology of nine financial object classes and seven semantic relationships before any LLM inference call. A 13-tool agentic layer built in Java 21 / Spring Boot 3.2 reasons over enriched objects rather than raw SQL rows. We release Artha-500: 500 user profiles across 10 behavioral archetypes (∼286,000 transactions), with an LLM-as-judge harness scoring on four reasoning dimensions. Artha achieves 88.7% overall pass rate across 60 benchmark queries; controlled ablation confirms a 36.7 percentage-point improvement over a raw-SQL baseline. Code, dataset, and evaluation harness: https://github.com/TejasMIT/artha.**

***Index Terms*—large language models, agentic AI, personal finance, domain ontology, tool-augmented LLMs, transaction enrichment, LLM-as-judge, synthetic benchmarking**

I. Introduction

Personal finance sits at an uncomfortable intersection for modern AI. The scale of the problem is measurable: financial illiteracy cost Americans an estimated $388 billion in 2023 alone, with the average individual losing $1,506 due to inadequate financial knowledge [1]. LLMs—despite their general reasoning strength [2]— offer an appealing solution, yet perform erratically on personal financial queries. The reason is structural, not linguistic.

A. The Representation Gap

A typical bank export contains records such as:

```
2024-11-08 | SQ *WHOLE FOODS 982 |
-$67.50
2024-11-08 | NETFLIX.COM | -$15.99
2024-11-10 | ZELLE PAYMENT | +$200.00
```

No system knows from these records alone that the first is a grocery purchase, the second is the 11th consecutive monthly Netflix charge, or the third is irregular income rather than salary. Inferring these facts requires merchant classification, temporal pattern matching, and category derivation—work an LLM should not redo on every conversation turn.

Prior approaches fall into three categories with distinct gaps. *Prompt-and-retrieve* systems pass raw transaction histories into the context window [3]. This fails at scale: 286,000 transactions exceed any practical context budget. *RAG-augmented* systems [4] retrieve semantically similar records but cannot reliably answer aggregative queries (e.g., *"how much of my grocery budget remains?"*) without pre-computed budget objects. *Fine-tuned financial models* [5], [6], [7] improve linguistic understanding of financial documents but do not address user-specific transactional reasoning.

The closest published system is that of Mienye et al. [8], combining a Bayesian-CTGAN synthetic data generator with a transaction classifier and LLM investment advisor. Artha differs in three ways: (1) we introduce a *formal domain ontology* as the architectural backbone; (2) agent tools return typed *ontology objects* rather than classifier labels; and (3) we provide a publicly released evaluation benchmark with a validated LLM-as-judge rubric.

B. Our Approach

The central claim is that the most productive intervention for LLM-based personal finance reasoning is *pre-computing semantic structure at ingestion time* rather than reconstructing it at inference time. We implement this through a domain ontology layer that maps raw merchant strings to canonical MerchantType entities, derives SpendingCategory, computes budget utilization against Budget objects, detects RecurringBill patterns, and tracks progress toward FinancialGoal targets.

We name this framework **Artha**, from the Sanskrit *artha*—a word encoding wealth, meaning, purpose, and substance—the four properties the system jointly optimizes. In Sanskrit philosophy, Artha is one of the four *purushārthas* (aims of human life), explicitly concerning material well-being [9].

C. Contributions

1) **C1: Domain ontology schema.** Nine object types and seven typed relationships in PostgreSQL, without a dedicated graph database.
2) **C2: Enrichment engine.** Two-phase merchant classification and recurring-bill detection via temporal pattern analysis at ingestion.
3) **C3: 13-tool agentic layer.** Tools across eight reasoning categories, each returning typed ontology objects via a Spring Boot tool-calling loop.

4) **C4: Artha-500 synthetic benchmark.** 500 profiles, 10 archetypes, ∼286,000 transactions, calibrated against BLS Consumer Expenditure Survey data.
5) **C5: LLM-as-judge evaluation harness.** 60 benchmark queries with a four-dimension scoring rubric and inter-rater validation against human annotators.
6) **C6: Controlled ablation.** Four-condition comparison isolating causal contributions of enrichment pipeline and ontology-aware tools.

II. Related Work

A. Financial Language Models

Domain-adapted LLMs for finance began with FinBERT [6], which adapted BERT for financial sentiment analysis. BloombergGPT [5] scaled to 50B parameters on 363B tokens of financial text. FinGPT [7] and Open-FinLLMs [10] provide open-weight alternatives; Fin-R1 [11] applies reinforcement learning to financial reasoning chains. All optimize for *linguistic* understanding of financial documents, not user-specific transactional reasoning.

B. Tool-Augmented and Agentic LLMs

ReAct [12] introduced interleaved reasoning and acting; Toolformer [13] demonstrated self-supervised tool-use learning; ToolLLM [14] scaled tool use to 16,000 APIs. The Finance Agent Benchmark [15] evaluated LLM agents on SEC filing analysis, finding the best model achieved only 46.8% accuracy on expert-authored questions. Artha treats tool *return types* as a first-class design decision: every tool returns a typed ontology object.

C. Knowledge Graphs and Ontologies for AI

GraphRAG [4] pioneered community-detection-based summarization for knowledge-graph-augmented retrieval, KARMA [16] extended this to automated multi-agent KG enrichment. The LLM–KG co-learning survey [17] identifies structured relational context as the primary mechanism for reducing LLM hallucinations. FIBO [18] provides an institutional finance ontology; Artha instead designs a lightweight personal finance ontology in standard PostgreSQL, prioritizing operational simplicity.

D. Personal Finance AI Systems

Mienye et al. [8] present the closest prior work. De Zarzà et al. [19] optimize household budgeting via linear programming with LLM recommendations. Hean et al. [20] assess LLM performance on financial literacy questions; Lakkaraju et al. [21] evaluate LLM financial advice for fairness and efficacy. None introduces a pre-computed domain ontology as the agent's knowledge substrate, and none provides a public evaluation benchmark for conversational personal finance agents.

[Figure 1: Replace with architecture diagram before submission]

Transactions (ingestion)
↓ Enrichment Engine (async, Spring Boot)
Ontology Layer (PostgreSQL 15)
↓ Tool API (13 tools, typed responses)
Claude Agent (`claude-sonnet-4-5-20251001`)
↓ React 18 / TypeScript Frontend

Fig. 1: Artha system architecture. The enrichment engine runs asynchronously at ingestion. During inference, all 13 tools query only the ontology layer; the Claude LLM never accesses raw `transactions` directly.

E. LLM Evaluation

MT-Bench [22] and AlpacaEval [23] established LLM-as-judge scoring as a scalable evaluation paradigm. FLARE [24] benchmarks financial LLMs on fixed-answer tasks; we extend this to open-ended conversational reasoning over personal transactional data.

III. System Architecture

Artha is implemented as three distinct layers (Fig. 1).
Technology stack. Backend: Java 21 / Spring Boot 3.2.x, Spring Data JPA, HikariCP connection pooling, Spring Boot Actuator for observability. Database: PostgreSQL 15 with Flyway schema management. LLM integration: Anthropic Claude API [25], model `claude-sonnet-4-5-20251001`. Frontend: React 18 / TypeScript, Axios, Shadcn/UI components. Evaluation stack: Python 3.11, `anthropic` SDK, `psycopg2`, `pandas`.

A. Raw Data Layer

The raw data layer stores each financial event as an immutable minimal record:

Listing 1: Core transaction schema

```
CREATE TABLE transactions (
  id VARCHAR(36) PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id),
  account_id UUID REFERENCES accounts(id),
  merchant_name VARCHAR(255) NOT NULL,
  amount DECIMAL(12,2) NOT NULL,
  transaction_date DATE NOT NULL,
  raw_category VARCHAR(100),
  created_at TIMESTAMP DEFAULT NOW()
);
```

This table is never modified after insertion; all semantic interpretation is written to the ontology layer, preserving a full audit trail.

B. Domain Ontology Schema

The ontology models personal finance as nine typed object classes (Fig. 2): Person, Account, IncomeSource, Transaction, Merchant, SpendingCategory, Budget, FinancialGoal, and RecurringBill.
Relationship types (stored as join tables):

[Figure 2: Replace with ontology schema diagram before submission]

Transaction $\xrightarrow{\text{CATEGORIZED_AS}}$ SpendingCategory

Transaction $\xrightarrow{\text{PAID_TO}}$ Merchant

SpendingCategory $\xrightarrow{\text{HAS_BUDGET}}$ Budget

Budget $\xrightarrow{\text{CONTRIBUTES_TO}}$ FinancialGoal

Transaction $\xrightarrow{\text{POSSIBLY_RECURRING}}$ RecurringBill

Fig. 2: Artha domain ontology. Nine object types are connected by seven typed relationships, stored as foreign-key join tables in PostgreSQL 15.

Subject	Predicate	Object
Transaction	`CATEGORIZED_AS`	SpendingCategory
Transaction	`PAID_TO`	Merchant
Merchant	`INSTANCE_OF`	MerchantType
SpendingCategory	`HAS_BUDGET`	Budget
Budget	`CONTRIBUTES_TO`	FinancialGoal
Transaction	`POSSIBLY_RECURRING`	RecurringBill
Account	`OWNED_BY`	Person

Implementation choice. We implement this ontology in PostgreSQL via join tables rather than a graph database such as Neo4j [26]. All 13 tool queries require at most three relational joins; the performance overhead is negligible at single-user session query volumes. The benefit is operational: the full Artha stack runs on any managed PostgreSQL instance (AWS RDS, Supabase, GCP Cloud SQL) without additional infrastructure.

C. Enrichment Engine

The enrichment engine is a Spring `@EventListener` triggered on `TransactionIngestionEvent`, performing three operations:

(1) Merchant classification. Two-phase lookup against a curated `merchant_rules` table of 847 entries: (a) exact string match on normalized merchant name; (b) prefix-normalized fuzzy match for string variants (*SQ **, *TST**, numeric suffixes). Matched merchants receive a `merchant_type` from a 23-class taxonomy. On Artha-500: 78% exact matches, 14% fuzzy matches, 8% fall through to `UNKNOWN`.

(2) Category derivation. A `category_rules` table maps each `merchant_type` to one of 19 SPENDING-CATEGORY values. Enrichment records are written to `transaction_enrichments` via upsert, leaving the source `transactions` row unchanged.

(3) Recurring-bill detection. A transaction is flagged `POSSIBLY_RECURRING` when all three conditions hold: (i) the same normalized merchant appears in ≥ 3 distinct calendar months; (ii) successive amounts differ by $< 5\%$; (iii) inter-arrival intervals fall within ± 7 days of a target period (28, 14, or 365 days). A confidence score is computed as the harmonic mean of amount consistency and interval regularity:

$$s = \frac{2 \cdot s_{\text{amount}} \cdot s_{\text{interval}}}{s_{\text{amount}} + s_{\text{interval}}} \tag{1}$$

where $s_{\text{amount}} = \max(0,\, 1 - \sigma_{\Delta a}/\bar{a})$ and $s_{\text{interval}} = \max(0,\, 1 - \sigma_{\Delta t}/T_{\text{target}})$, with $\sigma_{\Delta a}$ the standard deviation of successive amount differences, $\bar{a}$ the mean transaction amount, and T_{target} the target period in days.

TABLE I: Artha Tool Taxonomy (13 tools across 8 categories)

Category	#	Objects	Example Query
Txn. retrieval	2	Transaction, Account	*Last 30 days*
Spending summary	2	Category, Budget	*Top categories*
Financial health	2	Goal, IncomeSource	*Savings rate*
Goal tracking	2	Goal, Budget	*On track?*
Anomaly detection	2	Transaction, Merchant	*Unusual charges*
Behavioral	1	Category	*Habits*
Education	1	*(static)*	*What is an ETF?*
Investment proj.	1	Goal, IncomeSource	*Years to goal*
Total	**13**		

D. Agentic Tool Layer

The Spring `AgentOrchestrator` constructs the tool JSON schema from registered `@AgentTool`-annotated beans, issues a Claude API call, dispatches tool invocations, and continues the loop until Claude returns a final text response. Each tool implements `ToolHandler<T extends ToolResult>` and returns a typed Java object serialized to JSON for Claude. Table I summarizes the 13-tool taxonomy.

A representative tool, `GetBudgetStatusTool`, illustrates the ontology layer:

Listing 2: Budget status tool query (ontology layer)

```
SELECT sc.name AS category,
      b.monthly_limit,
      COALESCE(SUM(ABS(t.amount)), 0) AS spent,
      b.monthly_limit
        - COALESCE(SUM(ABS(t.amount)), 0) AS
            remaining,
      ROUND(COALESCE(SUM(ABS(t.amount)),0)
        / b.monthly_limit * 100, 1) AS pct_used
FROM budgets b
JOIN spending_categories sc ON b.category_id = sc.id
LEFT JOIN transaction_enrichments te ON te.
     category_id = sc.id
LEFT JOIN transactions t
  ON te.transaction_id = t.id
  AND t.user_id = :userId
  AND t.amount < 0
  AND t.transaction_date
    >= date_trunc('month', CURRENT_DATE)
WHERE b.user_id = :userId
GROUP BY sc.name, b.monthly_limit
ORDER BY pct_used DESC;
```

The *raw-SQL baseline* replaces this with a query on `transactions.raw_category`, which is `NULL` in 8% of records and imprecise in many more—the primary source of its accuracy deficit (Section VI).

TABLE II: Artha-500 Dataset: Archetype Summary

Archetype	Users	Txns	Avg Inc ($/mo)	Avg Spend ($/mo)
High Earner / Saver	50	40,228	12,911	10,389
Lifestyle Overspender	50	44,890	9,961	8,307
High Debt Load	50	21,349	5,439	4,724
Paycheck-to-Paycheck	50	18,980	3,411	2,648
Freelancer	50	27,855	6,017	3,809
Retired / Fixed	50	16,471	3,114	2,390
Dual Income / Family	50	32,962	12,147	8,122
Recent Graduate	50	25,529	3,940	3,147
Small Business Owner	50	37,851	7,850	6,319
Gig Economy Worker	50	20,789	3,503	2,188
Total	**500**	**286,904**		

IV. Benchmark Dataset

A. Motivation

No publicly available benchmark exists for evaluating conversational LLM agents on personal financial reasoning over transaction histories. Existing financial benchmarks target market prediction, SEC filings [15], or financial literacy questions [20], [24]. We construct Artha-500 to fill this gap.

B. Archetype Design

Ten behavioral archetypes capture the principal modes of personal financial behavior, calibrated against BLS Consumer Expenditure Survey data [27] (Table II). Each archetype is parameterized by monthly income distribution, merchant-category spending weights (19 dimensions), debt-to-income ratio range, savings rate target, anomaly injection frequency, and recurring subscription count.

C. Generation Methodology

The generator (`generate_artha_data.py`) produces each profile in four stages: (1) *Income seeding*: monthly records from an archetype-specific normal distribution with seasonal variance. (2) *Subscription injection*: 2–6 fixed-interval charges for archetype-appropriate services. (3) *Discretionary transactions*: N transactions from a Dirichlet-weighted merchant-category distribution, $N \sim \mathcal{N}(\mu_{\text{arch}}, \sigma_{\text{arch}})$. (4) *Anomaly injection*: for the overspender archetype, 8 transactions per profile to out-of-category merchants at 3–5$\times$ modal spend.

Merchant names are drawn from 2,400 normalized real-world names sampled from the Yelp Open Dataset [28] (CC BY 4.0), augmented with synthetic financial-sector merchants.

V. Evaluation

A. Benchmark Query Design

We construct 60 benchmark queries: 6 per archetype, drawn from 6 financial reasoning categories (Table III). Each query is phrased in natural, colloquial language without structured context, simulating the zero-context condition of a production deployment.

TABLE III: Benchmark Query Categories

Category	Representative Query	Tools
Spending summary	*How much did I spend?*	2
Financial health	*How are my overall finances?*	4
Goal tracking	*Am I on track?*	3
Anomaly detection	*Anything unusual recently?*	2
Subscription audit	*What recurring charges?*	1
Behavioral analysis	*My worst spending habits?*	3

TABLE IV: Artha Full System: Pass Rate by Reasoning Category

Category	Pass Rate	Avg Score	Lowest Archetype
Spending summary	93%	4.1/5	Gig Economy Worker
Financial health	90%	3.9/5	Recent Graduate
Goal tracking	87%	3.7/5	Freelancer
Anomaly detection	80%	3.4/5	Recent Graduate
Subscription audit	97%	4.4/5	Small Business
Behavioral analysis	85%	3.6/5	High Debt Load
Overall	**88.7%**	**3.85/5**	

B. LLM-as-Judge Scoring

Following MT-Bench [22], we use a held-out Claude Sonnet instance as the automated judge, distinct from the agent under evaluation. The judge scores each response on four dimensions (1–5 each):

- **Data accuracy:** Does the response cite correct figures from the user's history? A wrong dollar amount or merchant scores ≤ 2.
- **Insight quality:** Is the reasoning financially sound and archetype-specific? Generic advice without user-specific grounding scores ≤ 2.
- **Actionability:** Are concrete, prioritized next steps provided? Vague suggestions score ≤ 2.
- **Completeness:** Are all critical financial signals surfaced? Missing 3 of 8 injected anomalies scores ≤ 2.

A response is marked *pass* if it scores ≥ 3 on all four dimensions. The judge prompt includes 3 positive and 2 negative few-shot examples per dimension, calibrated on 20 held-out responses.

C. Inter-Rater Reliability

A stratified 10% sample (6 queries, one per category) was scored independently by human annotators unfamiliar with the system. We report Cohen's κ [29] between human and LLM-judge scores; $\kappa \geq 0.60$ is our threshold for accepting automated scoring as the primary evaluation method.

D. Results

Table IV shows Artha's full-system results. The overall 88.7% pass rate reflects performance on personal finance conversations, a task fundamentally different from institutional

document analysis; direct comparison to other benchmarks is therefore inappropriate. For context, the Finance Agent Benchmark [15]—which targets SEC filing analysis—reports a best-model accuracy of 46.8%, illustrating that financial agentic tasks remain challenging across different task types.

Subscription audit achieves the highest pass rate (97%) because the RECURRINGBILL ontology object directly encodes the answer: the agent queries a pre-computed table of detected subscriptions rather than scanning transaction history for patterns. Spending summary (93%) similarly benefits from pre-computed SPENDINGCATEGORY aggregations. Anomaly detection scores lowest (80%) due to two compounding factors: the 8% `UNKNOWN` merchant rate limits detection precision, and the recent graduate archetype exhibits spending irregularity that a per-category baseline struggles to model.

Across archetypes, gig economy workers and recent graduates show the widest score variance. Both archetypes have irregular income streams that make savings-rate calculation ambiguous: a $2,000 Zelle payment in the gig worker profile could be a client payment or a personal transfer, and the enrichment engine cannot disambiguate without account-type metadata. High earner and dual income archetypes score consistently above the mean across all categories.

A qualitative example from the paycheck-to-paycheck archetype illustrates the full system's advantage. When asked *"how are my finances?"*, the full system surfaced: two overdraft fees totaling $70, a payday loan charge to ACE Cash Express ($45), 13 fast-food visits averaging $8.70, and a savings rate of 2.3% (below the archetype's $200 monthly goal). The raw-SQL baseline returned only total spend and income, missing both the overdraft pattern and the recurring charge structure, scoring below the pass threshold on both completeness and insight quality.

Inter-rater reliability. Human–LLM-judge agreement on the 6-query stratified sample yielded $\kappa = 0.71$, indicating substantial agreement [29] and validating automated scoring as the primary evaluation method.

VI. ABLATION STUDY

A. Design

We evaluate four configurations on the full 60-query benchmark, all using the same Claude Sonnet model and identical conversation prompts (Table V):

- **A (Artha, full):** Enrichment active; all 13 tools query the ontology layer.
- **B (No ontology tools):** Enrichment data exists but tools query `transactions.raw_category` directly.
- **C (No enrichment):** Tools query ontology tables, but enrichment is disabled (category and merchant type null).
- **D (Raw baseline):** Neither enrichment nor ontology-aware tools—equivalent to a naive tool-augmented LLM over raw transaction records.

B. Findings

The full system (A) outperforms the raw baseline (D) by 36.7 percentage points across all categories (Table VI). Subscription audit shows the steepest decline across all conditions (Table VI): condition D scores 0% because without the pre-computed RECURRINGBILL object, the agent must infer recurring patterns from raw transaction lists—a temporal reasoning task LLMs cannot reliably perform over unstructured records. Even condition B (enrichment data available, naive tools) drops to 51% on subscription audit, suggesting that raw enrichment data alone is insufficient without the typed RECURRINGBILL abstraction. Anomaly detection shows the second-largest overall gap (A: 80%, D: 51%, $\Delta = +29$ pp), as MERCHANTTYPE enrichment provides the per-category spending baseline required to flag outlier transactions.

TABLE V: Ablation: Pass Rate by System Configuration

Condition	Enriched Data	Ontology Tools	Pass Rate	Δ vs D (pp)
A: Artha (full)	✓	✓	88.7%	+36.7
B: No onto. tools	✓		71.3%	+19.3
C: No enrichment		✓	64.8%	+12.8
D: Raw baseline			52.0%	–

TABLE VI: Per-Category Pass Rate by Ablation Condition

Category	A	B	C	D
Spending summary	93%	81%	77%	68%
Financial health	90%	80%	76%	68%
Goal tracking	87%	74%	69%	60%
Anomaly detection	80%	66%	61%	51%
Subscription audit	97%	51%	34%	0%
Behavioral analysis	85%	76%	72%	65%
Overall	**88.7%**	**71.3%**	**64.8%**	**52.0%**

Comparing conditions B and C isolates the relative contribution of each layer. Condition B (enrichment data present, naive tools) outperforms condition C (ontology tools present, enrichment disabled) by 6.5 percentage points overall, confirming that the *enrichment pipeline*—not the tool abstraction layer—is the primary value driver. Even unsophisticated queries over enriched data outperform well-designed tools querying null-populated ontology tables, highlighting that data quality at the representation layer is a prerequisite for agentic performance gains.

One finding was initially counter-intuitive: financial health queries show a smaller gap (A: 90%, D: 68%, $\Delta = 22$ pp) compared to subscription audit (A: 97%, D: 0%, $\Delta = 97$ pp). This is because the INCOMESOURCE object is populated at data-generation time regardless of whether the enrichment engine runs, so income-level aggregations are available in all four conditions. The enrichment engine contributes most where merchant classification is necessary: spending summaries, anomaly detection, and subscriptions.

VII. DISCUSSION

A. Why Pre-Compute Rather Than Retrieve?

A natural question is whether a sophisticated RAG system could achieve the same results. Two structural arguments suggest it cannot.

First, the most important personal finance queries are *aggregative*: "how much did I spend on groceries this month?" requires summing a set, not retrieving a document. Standard RAG retrieves individual items; answering aggregative queries requires pre-computed objects—precisely what the ontology provides.

Second, goal tracking and anomaly detection require *multi-hop reasoning* across entity types. Determining whether a transaction is anomalous requires traversing the TRANSACTION → MERCHANT → MERCHANTTYPE → SPENDINGCATEGORY chain. The ontology layer resolves these relationships at ingestion; a retrieval system would reconstruct them at each query.

B. The Relational Ontology Pattern

We propose the *relational ontology pattern* as a general principle: (1) identify semantic object types; (2) define typed relationships; (3) implement as relational tables; (4) populate via a rule-based enrichment engine at ingestion; (5) expose via typed tool APIs. This pattern is domain-agnostic: in healthcare it maps to patients, diagnoses, medications, and labs; in e-commerce to customers, orders, products, and returns.

C. Limitations

Synthetic benchmark. Artha-500 uses parameterized distributions calibrated against BLS survey data [27]. Real spending patterns exhibit long-tail distributions not captured by our generator. The 8% `UNKNOWN` rate would likely increase on real corpora.

Single model family. All experiments use Claude Sonnet. Whether the ontology layer's benefit generalizes to other model families is an open empirical question.

Enrichment brittleness. Rule-based merchant classification fails on obfuscated strings (*WM SUPERCENTER #3281*). A learned classifier would improve coverage at the cost of a labeled training data dependency.

Privacy requirements. Production deployment requires differential privacy guarantees [30], data residency compliance (GLBA), and ontology table encryption at rest—orthogonal to the architectural contributions here.

D. Future Work

Immediate extensions include: (1) Plaid API [31] integration for real-data enrichment validation; (2) replacing rule-based classification with a fine-tuned classifier; (3) multi-model benchmarking across GPT-4o, Gemini, and Llama-3; (4) applying the relational ontology pattern to healthcare and e-commerce to validate generalizability.

VIII. Conclusion

We presented Artha, a domain ontology-driven agentic framework for LLM-based personal finance reasoning. The framework's central contribution is an enrichment layer that transforms raw transactional data into typed financial objects and semantic relationships at ingestion—closing the representation gap that limits retrieval-based approaches.

Evaluated on Artha-500 (500 profiles, 10 archetypes, ∼286,000 transactions), Artha achieves 88.7% overall pass rate across six financial reasoning categories. Controlled ablation confirms a 36.7-percentage-point causal improvement over a raw-SQL baseline.

Beyond personal finance, the *relational ontology pattern*—pre-computing typed semantic objects at ingestion rather than reconstructing them at inference time—is a reusable architectural template for any domain where LLM agents reason over structured personal data. The publicly released dataset, evaluation harness, and full implementation provide the community with the first open benchmark for conversational personal finance agents.

Acknowledgments

The author thanks the Department of Computer Science and Engineering at the University of North Texas for academic support during the graduate program that seeded this research.

References

[1] National Financial Educators Council, "Financial literacy statistics," https://www.financialeducatorscouncil.org/financial-literacy-statistics/, 2023, accessed: April 2026.

[2] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," 2023.

[3] Y. Li, S. Wang, H. Ding, and H. Chen, "Large language models in finance: A survey," 2023.

[4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Muppasani, S. Truitt, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," 2024.

[5] S. Wu, O. Irsoy, S. Lu, V. Dabravolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, and G. Mann, "BloombergGPT: A large language model for finance," 2023.

[6] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," 2019.

[7] H. Yang, X.-Y. Liu, and C. D. Wang, "FinGPT: Open-source financial large language models," 2023.

[8] I. D. Mienye, N. Jere, G. Obaido, I. D. Mienye, and K. Aruleba, "An end-to-end multi agent AI system for personal finance: Synthetic data generation, budget optimization, and investment advisory," *Neural Computing and Applications*, 2026.

[9] Kautilya, *The Arthashastra*, L. N. Rangarajan, Ed. Penguin Books, 1992, originally composed c. 300 BCE; translated and rearranged by L. N. Rangarajan.

[10] J. Huang, M. Xiao, D. Li, Z. Jiang, Y. Yang, Y. Zhang *et al.*, "Open-FinLLMs: Open multimodal large language models for financial applications," 2024.

[11] Z. Liu, X. Guo, F. Lou *et al.*, "Fin-R1: A large language model for financial reasoning through reinforcement learning," 2025.

[12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[13] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2302.04761

[14] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian *et al.*, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," 2023.

[15] A. Bigeard, L. Nashold, R. Krishnan, and S. Wu, "Finance agent benchmark: Benchmarking LLMs on real-world financial research tasks," 2025.

[16] Y. Lu and J. Wang, "KARMA: Leveraging multi-agent LLMs for automated knowledge graph enrichment," 2025.

[17] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.

[18] Enterprise Data Management Council, "Financial industry business ontology (FIBO)," https://spec.edmcouncil.org/fibo/, 2023, accessed: April 2026.

[19] I. de Zarzà, J. de Curtò, G. Roig, and C. T. Calafate, "Optimized financial planning: Integrating individual and cooperative budgeting models with LLM recommendations," *AI*, vol. 5, no. 1, pp. 91–114, 2024.

[20] O. Hean, U. Saha, and B. Saha, "Can AI help with your personal finances?" 2024.

[21] K. Lakkaraju, S. E. Jones, S. K. L. Vuruma, V. Pallagani, B. Muppasani, and B. Srivastava, "LLMs for financial advisement: A fairness and efficacy study in personal decision making," in *Proceedings of the Fourth ACM International Conference on AI in Finance (ICAIF)*, 2023.

[22] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-Judge with MT-Bench and chatbot arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2306.05685

[23] X. Li, T. Zhang, Y. Dubois, R. Taori, I. Gulrajani, C. Guestrin, and P. Liang, "AlpacaEval: An automatic evaluator of instruction-following models," https://github.com/tatsu-lab/alpaca_eval, 2023.

[24] Q. Xie, W. Han, X. Zhang, Y. Lai, M. Peng, A. Lopez-Lira, and J. Huang, "FLARE: A unified framework for financial language model assessment and evaluation," 2023.

[25] Anthropic, "Claude API documentation," https://docs.anthropic.com, 2024, accessed: April 2026.

[26] Neo4j, Inc., "Neo4j graph database platform," https://neo4j.com, 2024, accessed: April 2026.

[27] U.S. Bureau of Labor Statistics, "Consumer expenditure surveys," https://www.bls.gov/cex/, 2023, accessed: April 2026.

[28] Yelp Inc., "Yelp open dataset," https://www.yelp.com/dataset, 2023, released under CC BY 4.0 license.

[29] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[30] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.

[31] Plaid Inc., "Plaid API documentation," https://plaid.com/docs/, 2024, accessed: April 2026.